Ziele dieser Semesterwoche 4

- Sie verstehen die Funktionsweise von einer/einem:
  - *Priority-Queue*
  - *Heap*
  - *Adaptable Priority-Queue*

- Sie können eine *heap-basierte Priority-Queue* implementieren.

Priority Queue

Eine *Priority Queue* speichert eine Collection von *Entries*.
Entry: *Key/Value* -Paar.

Die wichtigsten Methoden:

- ***insert(k, v)*** fügt eine neue *Entry(k, v)* in die *Priority Queue* ein.
- ***removeMin()*** entfernt die Entry mit dem *kleinsten Schlüssel (Key)* und gibt diese Entry zurück.

Zusätzliche Methoden:

- ***min()*** liefert die Entry mit dem kleinsten Schlüssel, ohne diese zu entfernen.
- ***size()***
- ***isEmpty()***

Anwendungen: Auktionen, Börse, Warteschlangensysteme, etc.

Beispiel

Method	Return Value	Priority Queue Contents
insert(5,A)		{ (5,A) }
insert(9,C)		{ (5,A), (9,C) }
insert(3,B)		{ (3,B), (5,A), (9,C) }
min( )	(3,B)	{ (3,B), (5,A), (9,C) }
removeMin( )	(3,B)	{ (5,A), (9,C) }
insert(7,D)		{ (5,A), (7,D), (9,C) }
removeMin( )	(5,A)	{ (7,D), (9,C) }
removeMin( )	(7,D)	{ (9,C) }
removeMin( )	(9,C)	{ }
removeMin( )	null	{ }
isEmpty( )	true	{ }

Hinweis:
Der interne Aufbau der Priority-Queue kann auch anders sein (als hier dargestellt mit einer sortierten Liste).
Relevant ist, dass mit *removeMin()* das kleinste Element zurückgegben wird.

Sortieren mithilfe einer Priority Queue

- Mithilfe einer Priority Queue kann eine Menge aus vergleichbaren Elementen sortiert werden:
  1. Einfügen aller Elemente mithilfe von ***insert***
  2. Entfernen aller Elemente in sortierter Reihenfolge, mithilfe von ***removeMin*** Operationen.

- Das Laufzeitverhalten für diese Art zu Sortieren ist abhängig von der Implementierung der Priority Queue.

Algorithmus ***PQ-Sort(S, C)***

Input Sequence ***S***, Comparator ***C*** für die Elemente von ***S***

Output Sequence ***S*** sortiert in aufsteigender Reihenfolge gemäss ***C***

```
P ← Priority Queue mit
      Comparator C
while ¬ S.isEmpty()
    e ← S.remove(S. first ())
    P.insert(e, null)
while ¬ P.isEmpty()
    e ← P.removeMin().key()
    S.addLast(e)
```

Sequenz-basierte Priority Queue

Implementierung mit einer unsortierten Liste:



Performance:

insert() benötigt ***O(1)*** Zeit, da die Entry am Anfang oder am Ende der Sequenz eingefügt werden kann.

removeMin() und *min()* benötigen ***O(n)*** Zeit, da die ganze Sequenz durchlaufen werden muss, um das Minimum zu finden.

Entspricht *Selection-Sort* !

Implementierung mit einer sortierten Liste:



Performance:

insert() benötigt ***O(n)*** Zeit, weil wir die Stelle finden müssen, an der wir das Item einfügen müssen.

removeMin() und *min()* benötigen ***O(1)*** Zeit, da der kleinste Schlüssel sich am Anfang befindet.

Entspricht *Insertion-Sort* !

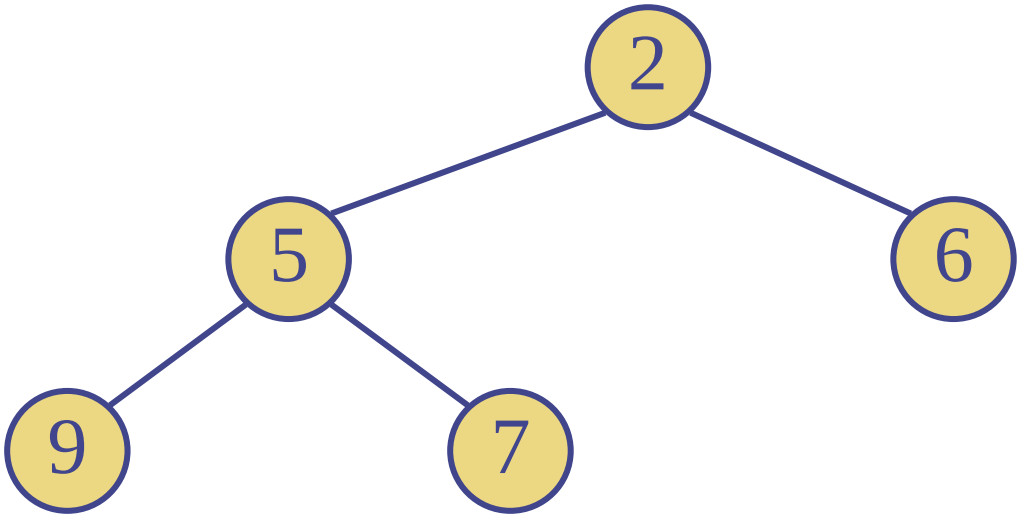
Heap-basierte Priority Queue

Ein *Heap* (Haufen, Halde) ist ein Binärbaum, welcher in seinen Knoten Schlüssel speichert und die folgende Eigenschaft besitzen:

Heap-Ordnung :

für jeden Knoten v, welcher nicht Wurzel ist, gilt:

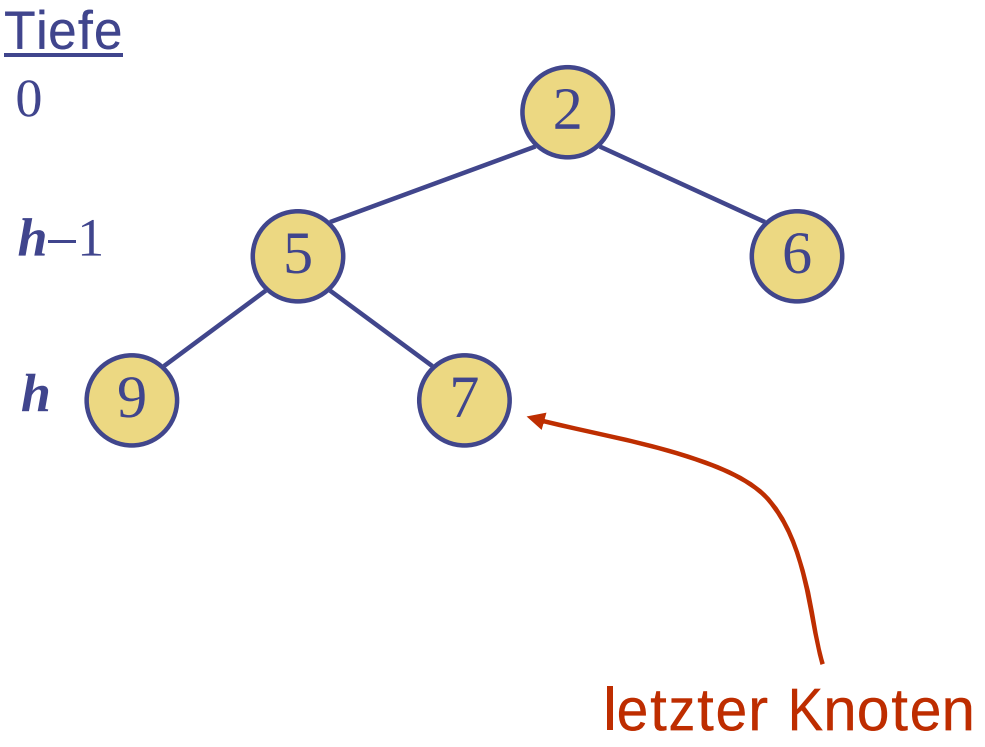
key(v) ≥ key(parent(v))



Eigenschaften eines Heaps

Ein Heap ist ein Binärbaum und hat folgende zusätzlichen Eigenschaften gegenüber einem normalen Binärbaum (sei *h* die Höhe des Heaps):

- für Tiefe i = 0, … , h-1 es sind 2^i Knoten auf der Tiefe *i* vorhanden (volle Levels)
- Auf der Tiefe *h* befinden sich die Knoten links (es wird von links her aufgefüllt).
- Der letzte Knoten eines Heaps ist der am weitestens rechts stehende Knoten auf der Tiefe *h* und wird speziell gemerkt.



- Es gibt maximal ein Knoten mit nur einem Kind, das ein linkes Kind sein muss.

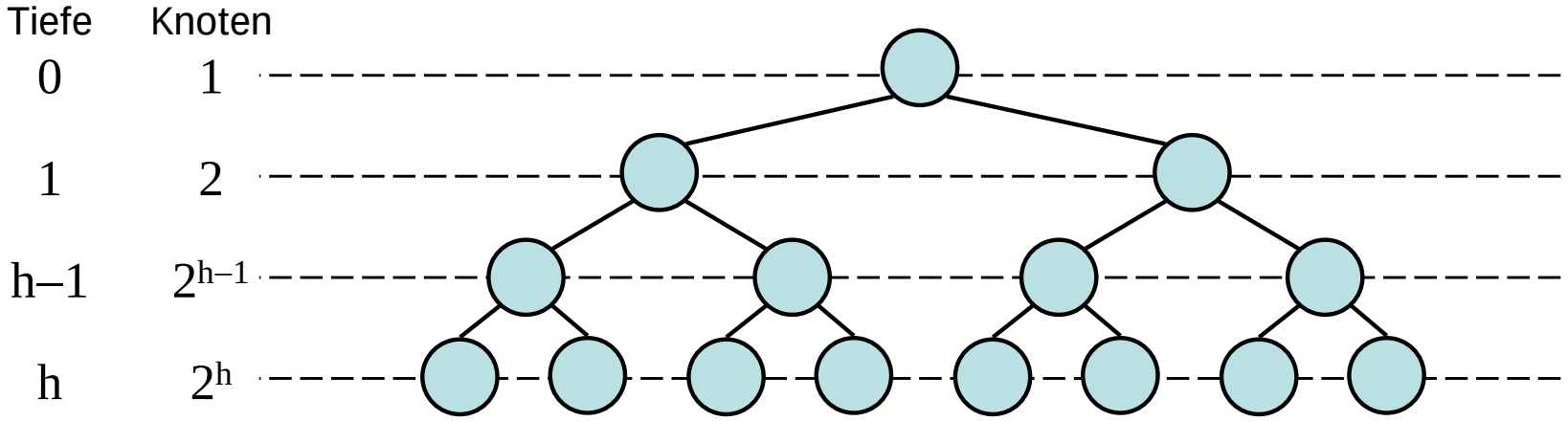
Höhe eines Heaps

Behauptung :

Ein Heap mit *n* Knoten besitzt eine Höhe *log n* .

Beweis :

- Sei *h* die Höhe eines vollen Heaps mit *n* Knoten:
  - $n = 1 + 2 + 4 + \ldots + 2^h = 2^{h+1} - 1$ *(Proposition A.13: geometric sum)*
  - $h = \log(n+1) - 1 \quad \Rightarrow \quad h \in O(\log n)$

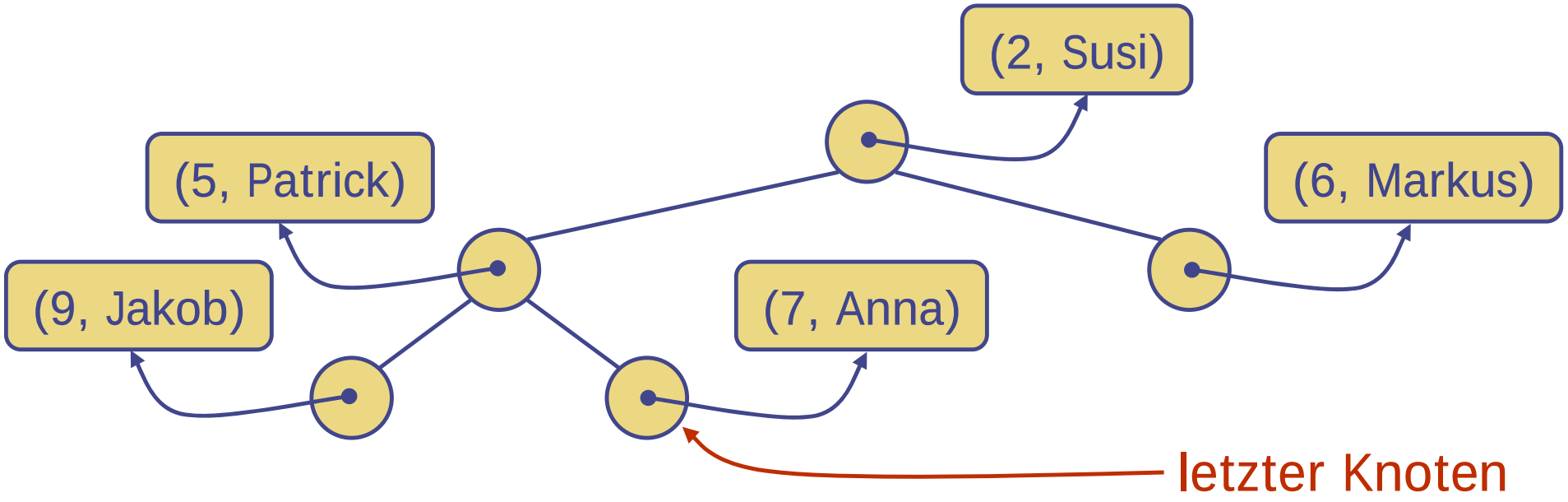


→ Die *Höhe h* eines Heaps ist ***O(log n)***

Heaps und Priority Queues

Mithilfe eines *Heaps* können wir eine *Priority Queue* implementieren:

1. Wir speichern eine *Entry<Key, Value>* an jedem Knoten des Heaps.
2. Der ***Root-Knoten*** enthält den *kleinsten* Key :
   → ***removeMin()*** resp. ***min()***



Bemerkung: Im Gegensatz zum obigen Beispiel wird oft nur der Key notiert (ohne Value).

Entries in einen Heap einfügen

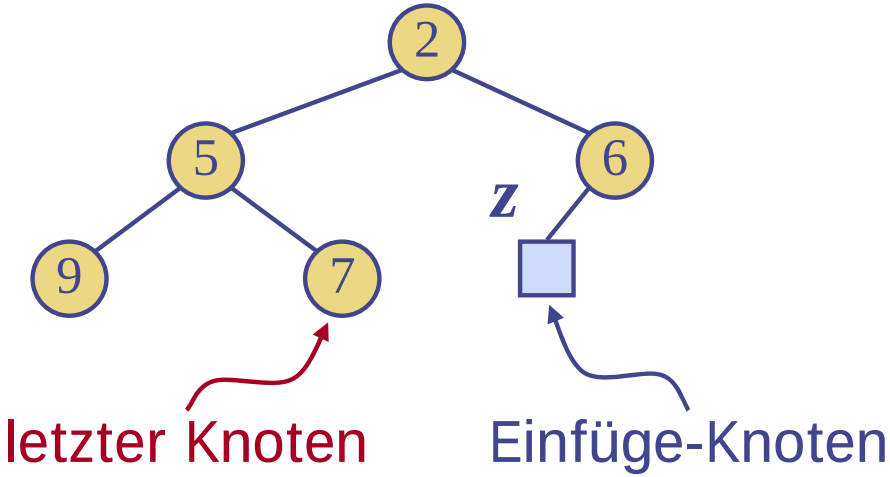
Die Methode **insert()** der Priority Queue entspricht dem Einfügen eines Schlüssels *k* in den Heap.

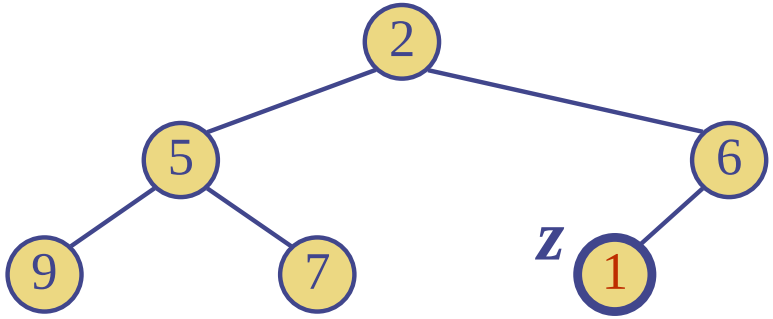
Der Einfüge-Algorithmus besteht aus drei Schritten:

1. Zuerst muss der Einfügeknoten *z* gefunden werden:
   → nächste Position ab dem letzten Knoten

2. Dann wird der Schlüssel *k* in *z* abgespeichert.

3. Und schliesslich muss die *Heap-Ordnung* überprüft und gegebenfalls wieder hergestellt werden (nächste Folie).

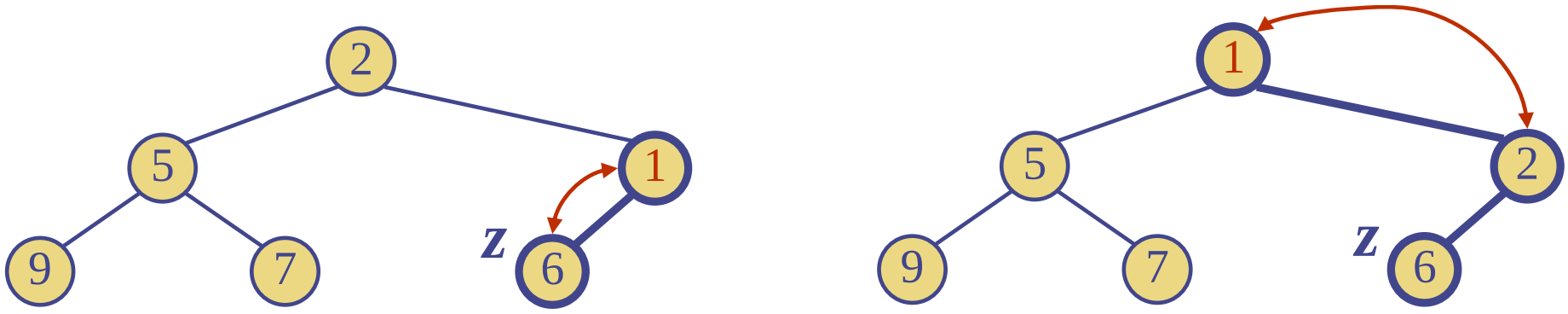




Upheap : Wiederherstellen der Heapeigenschaft aufwärts

Nach dem Einfügen eines neuen Schlüssels *k* könnte die *Heap-Ordnung* verletzt sein:

Der Algorithmus ***upheap*** stellt die Heap-Ordnung wieder her, indem der neue Schlüssel *k* entlang einem Pfad vom Einfügeknoten in Richtung Wurzel mit dem jeweils darüberliegenden Knoten vertauscht wird solange jener einen *grösseren* Schlüssel hat oder die *Wurzel* erreicht ist.



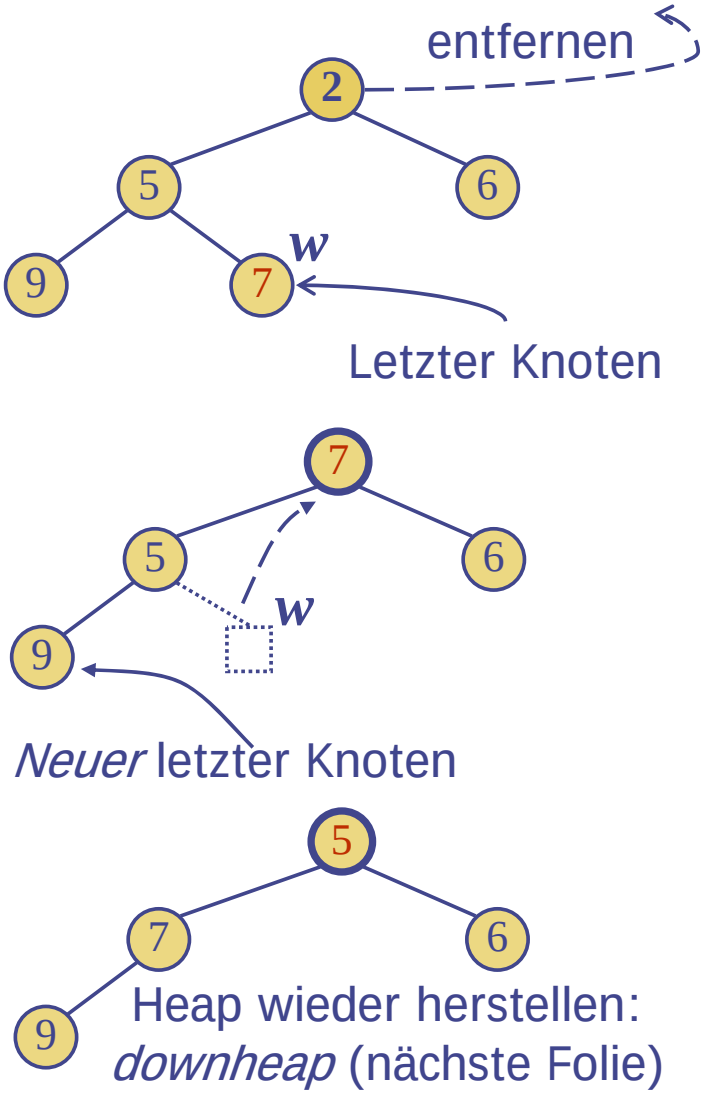
Da der Heap eine *Höhe* von *O(log n)* hat, benötigt *upheap* ***O(log n)*** Zeit.

Entfernen aus einem Heap

Die Methode **removeMin()** der *Priority Queue* entspricht dem entfernen der *Wurzel* eines *Heaps*.

Der **removal** Algorithmus besteht aus folgenden Schritten:

1. Entfernen des Schlüssels in der Wurzel (Minimum)
2. Ersetzen des Schlüssels in der Wurzel durch den Schlüssel des letzten Knotens *w* (es ist einer der Grössten)
3. Entfernen des letzten Knotens *w*
4. Wiederherstellen der Heap-Ordnung (nächste Folie)



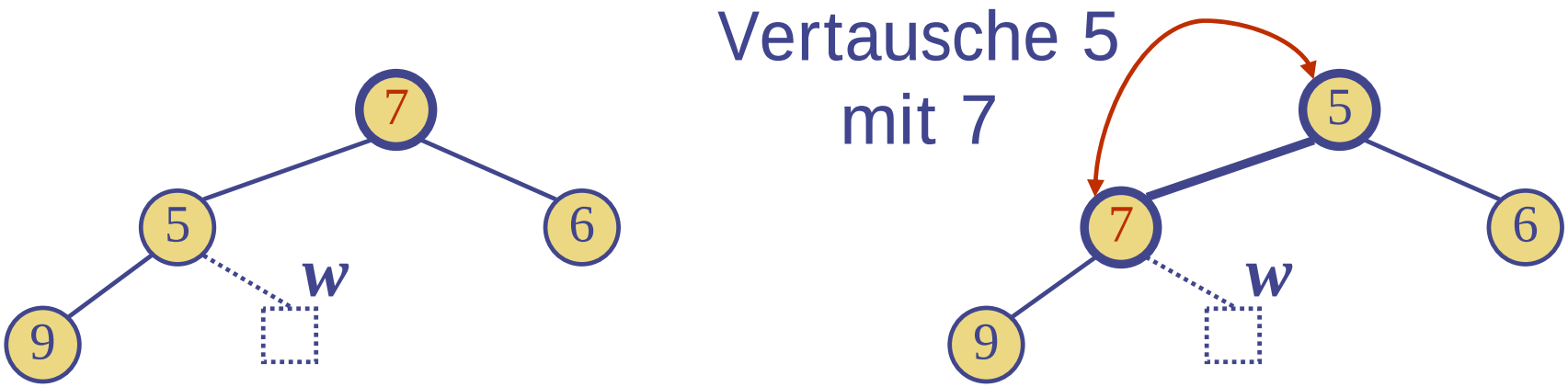
Downheap : Wiederherstellen der Heapeigenschaft abwärts

Durch Ersetzen des Schlüssels in der Wurzel durch den Schlüssel *k* des letzten Knotens wird die Heap-Ordnung im Normallfall verletzt.

Der Algorithmus ***downheap*** stellt die Heap-Ordnung wieder her, indem der Schlüssel *k* entlang dem Pfad von der Wurzel abwärts in Richtung des jeweils kleineren Schlüssel vertauscht wird.

downheap endet, sobald:

- eine Knoten-Position erreicht ist, dessen Kind-Schlüssel alle grösser oder gleich dem Schlüssel *k* sind.
- oder der tiefste Level erreicht ist.



Da der Heap eine *Höhe* von *O(log n)* hat, benötigt *downheap* ***O(log n)*** Zeit.

Heap-Sort

Mithilfe einer *Priority Queue* kann eine Sequenz von *n* Elementen sortiert werden:

1. alle Elemente mit ***insert()*** einfügen
2. alle Elemente wieder mit ***removeMin()*** auslesen

→ ausgelesene Sequenz ist sortiert ☺

Wenn die Priority Queue *heap-basiert ist:*

1. Einfügen: n * *insert()* = n * log n
2. Auslesen: n * *removeMin()* = n * log n

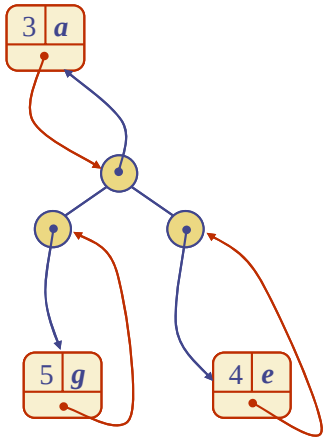
Total = 2n log n

→ Laufzeit: **O(n log n)**

Der Algorithmus ist bekannt unter dem Namen: ***Heap-Sort***

Adaptable Priority Queue

Bei einer *Adaptierbare Priority Queue* können *Schlüssel (Keys)* und *Werte (Values)* nach dem Einfügen nachträglich noch verändert werden (und die interne Ordnung wird wenn nötig intern angepasst).



Die wichtigsten Methoden:

insert(k, v):e	fügt eine neue *Entry(k, v) e* ein und gibt diese Entry zurück.
remove(e):e	entfernt eine *Entry e* und gibt diese zurück.
replaceKey(e,k):k	der Schlüssel der *Entry e* wird durch *k* ersetzt und der alte Schlüssel wird zurück gegeben. (die interne Ordnung wird intern aufrechterhalten)
replaceValue(e,v):v	der *Wert v* der *Entry e* wird durch *v* ersetzt und der alte Wert wird zurück gegeben.

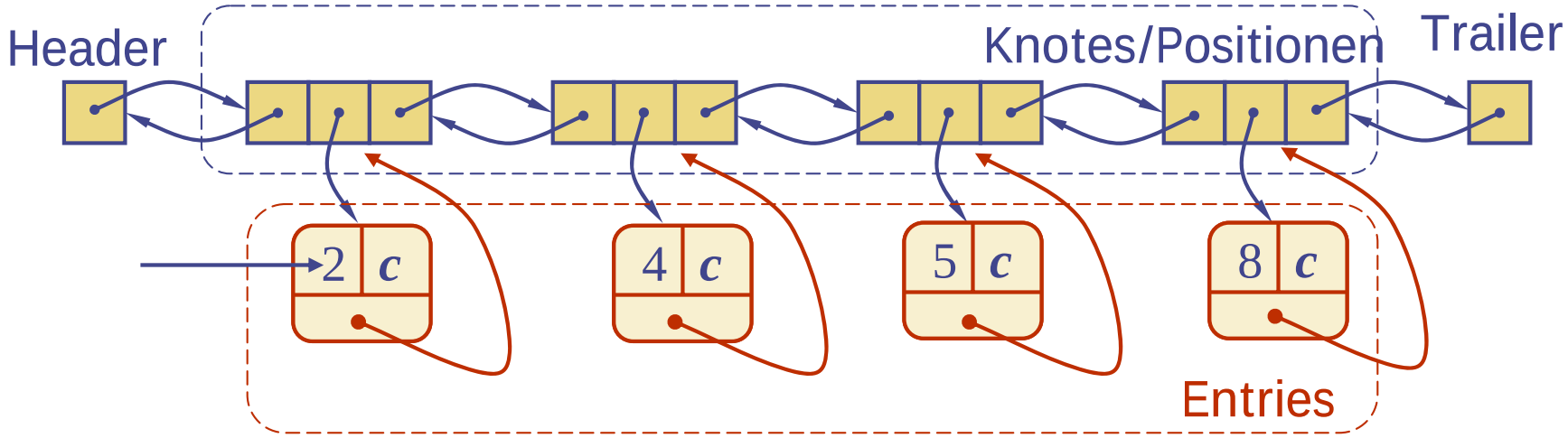
Beispiel
Insertion-Sort basierte Adaptable Priority Queue:

Operation	***Entry / Output***	*Adaptable PQ*
insert$(5,A)$	e_1	**$(\underline{5,A})$**
insert$(3,B)$	e_2	**$(\underline{3,B})$**$,(5,A)$
insert$(7,C)$	e_3	$(3,B),(5,A),$**$(\underline{7,C})$**
min()	e_2	**$(\underline{3,B})$**$,(5,A),(7,C)$
key(e_2)	3	**$(\underline{3,B})$**$,(5,A),(7,C)$
remove(e_1)	e_1	$(3,B),(7,C)$
replaceKey$(e_2,9)$	3	$(7,C),$**$(\underline{9,B})$**
replaceValue(e_3,D)	C	**$(\underline{7,D})$**$,(9,B)$
remove(e_2)	e_2	$(7,D)$

Listen basierte Implementierung

Eine *Adaptable Priority Queue* verwendet intern sog. *Location-Aware Entries* (*Lokations-bewusste Entries*):

- Die Entry kennt ihre Position in der dahinterliegenden Datenstruktur (Liste, Array, Heap, etc.).
- Z.B. mit *Back-Referenzen, Indizes,* etc.



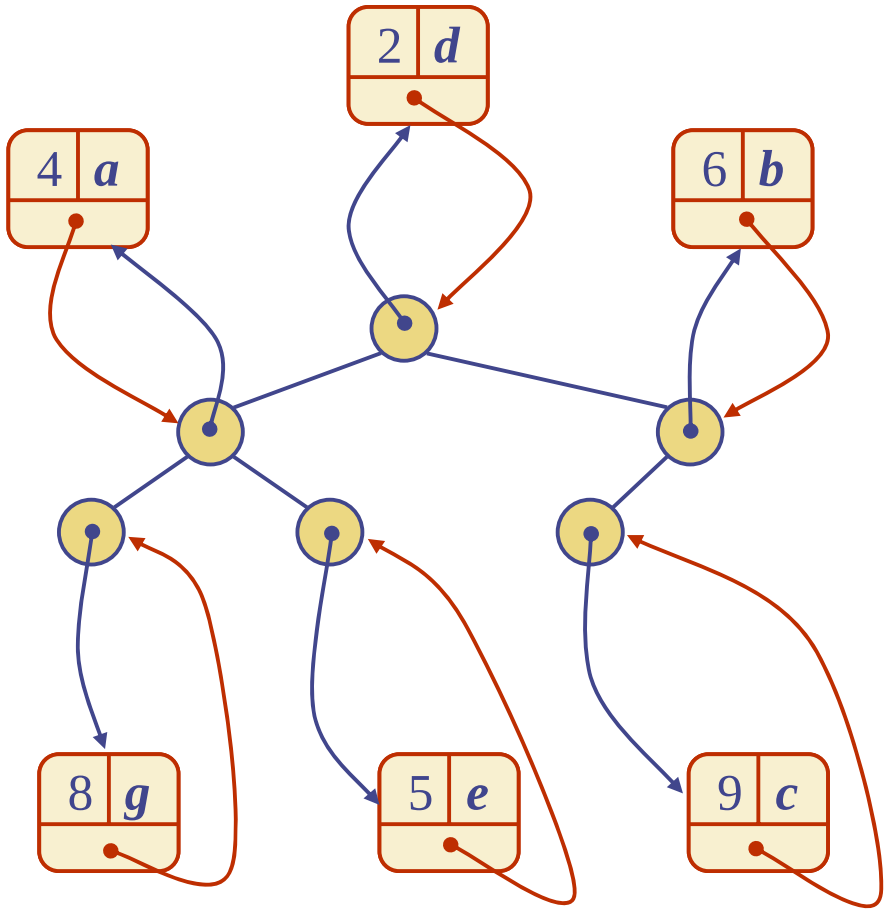
Wichtig: Back-Referenzen oder Indizes werden während allfälligen Swaps aktualisiert.

Heap basierte Implementierung

Bei ***replaceKey(e,k)*** wird nach dem ersetzen des Schlüssel ab der gegenwärtigen Position ein ***up-*** resp. ***downheap*** durchgeführt.

Da die entsprechenden Knoten in *O(1)* gefunden werden, wird die Laufzeit der gesamte Operation durch *up-* resp. *downheap* bestimmt:

→ ***O(log n)***



Performance

Mit Lokations-bewussten Entries können wir die Performance verbessern
(*rot/kursiv* : verbesserte Performance wegen dem Lokations-Bewusstsein):

Methode	**Unsortierte Liste**	**Sortierte Liste**	**Heap**
size, isEmpty	O(1)	O(1)	O(1)
insert	O(1)	O(n)	O(log n)
min	O(n)	O(1)	O(1)
removeMin	O(n)	O(1)	O(log n)
remove	***O(1)***	***O(1)***	***O(log n)***
replaceKey	O(1)	O(n)	***O(log n)***
replaceValue	O(1)	O(1)	O(1)